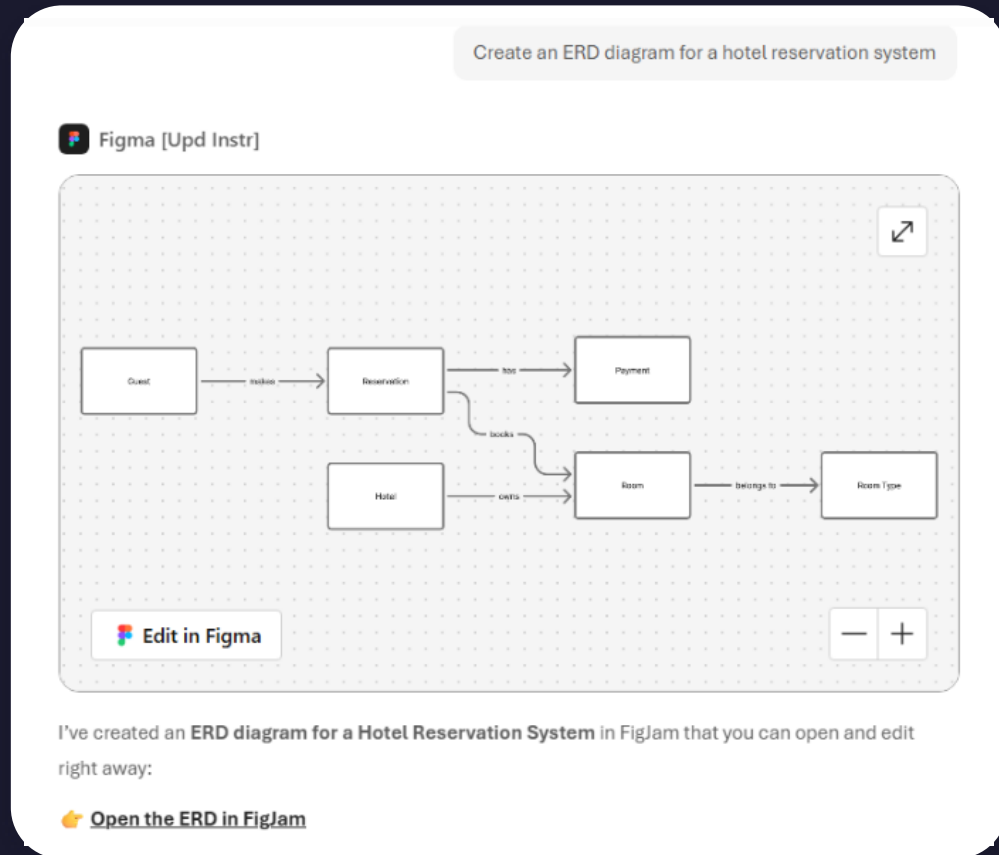
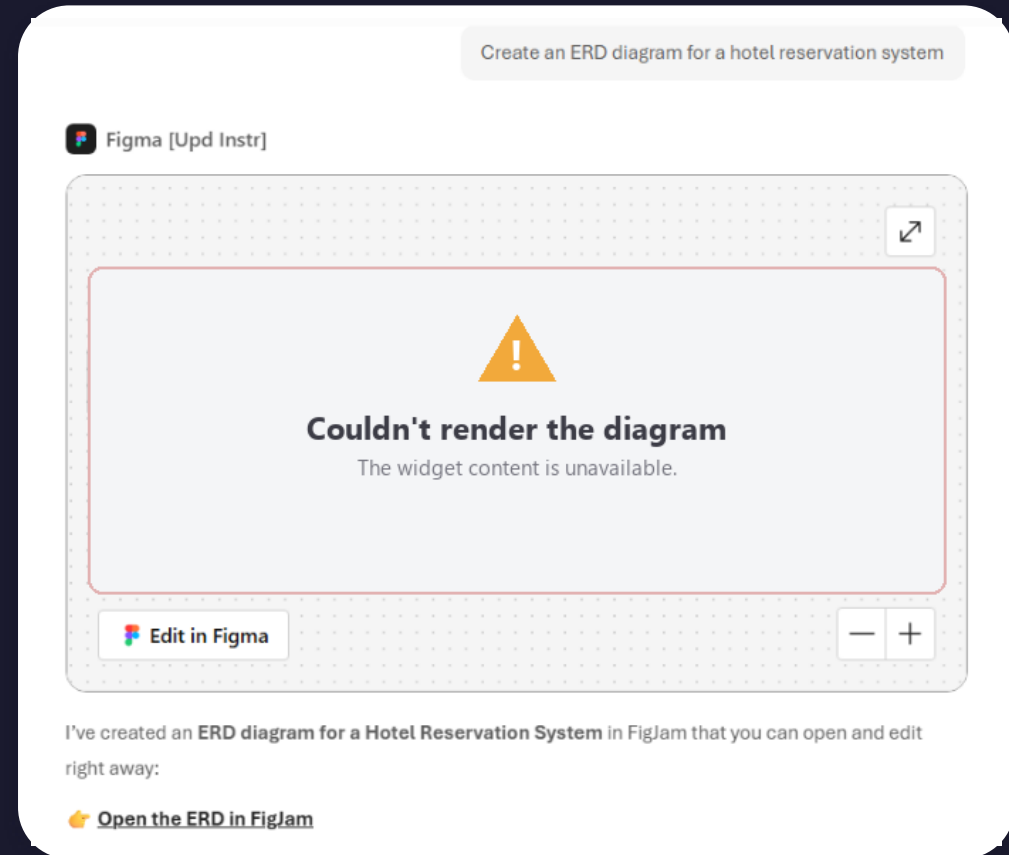


MCP Apps let 3rd-party apps render live widgets — charts, diagrams, cards — inside an AI agent's reply.

✓ widget renders



✗ widget fails — same query, same answer



200 OK. **Blank box.**

The tool call fired, the text answer is right — only the **rendered widget** broke. And not one automated check could **see** it.

VISUAL UX EVALUATION FOR AGENTIC AI

AI Builds the UI — Who Judges It?

Agent Skills that turn a coding assistant into a visual UX testing orchestrator

Tezan Sahu

tezansahu@microsoft.com

Sudipta Das

sudiptad@microsoft.com

We test the components — not the composition

UX component	Traditional software	Agentic AI
Layout (template)	Fixed, pre-designed	Dynamic — agent-generated / 3P templates
Content (data)	Static / deterministic	Non-deterministic — LLM-synthesized
Client (renderer)	Fixed	Fixed — functionally testable

The defect isn't in any one box.

The client is testable. But the **composed** experience (**Layout × Content × Client**) emerges only at runtime, and **that** is what nothing evaluates.

Any user experience is **Layout × Content × Client**.

In agentic AI, two of the three turn non-deterministic.

Three families of tools — all blind to the screen

Agent / LLM frameworks

HELM, AgentBench, LLM-as-judge

What it sees

Judge text & tool calls

✗ Never render the UI

Visual-regression tools

Applitools, Percy

What it sees

Pixel-diff against a baseline

✗ No baseline for non-deterministic UIs

Observability platforms

Braintrust, LangSmith

What it sees

Trace API calls & spans

✗ Never see what the user sees

The gap falls between AI evaluation and functional testing — no tool judges the rendered visual outcome.

What is an Agent Skill?

<skill-name>/ *(just a folder)*

SKILL.md the playbook · name + description + steps

references/ guides & rubrics

assets/ templates, reusable prompts

scripts/ deterministic helpers

A folder with a markdown file

- The description says when & why to load it
- The body holds step-by-step instructions.

Progressive disclosure

- Name + Description always loaded (cheap)
- The body loads only on a match
- Sub-files load only when a sub-task needs them — so it scales to many skills.

An open standard

- Introduced by Anthropic (Oct 2025)
- Spec at agentskills.io.
- Now read by GitHub Copilot, Claude Code, Cursor.
- Portable — no lock-in.

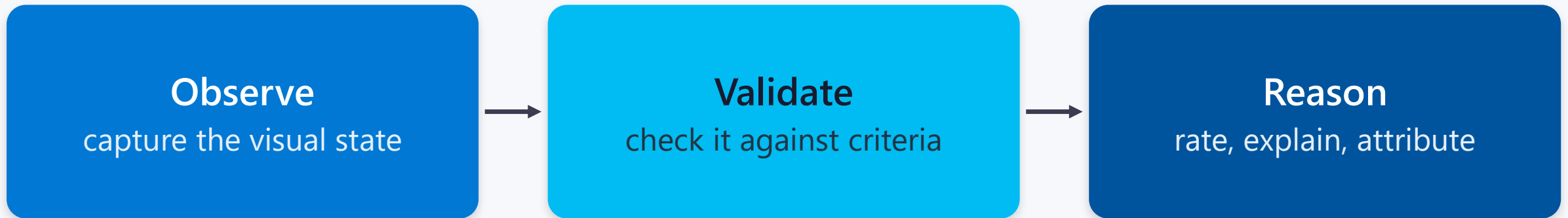
Install a skill → a general coding assistant gains a new, specialized capability.

Encode a human evaluator's loop as a skill

"How does a human evaluate one of these screens?"

Observe → Validate → Reason.

We package that loop as a skill.

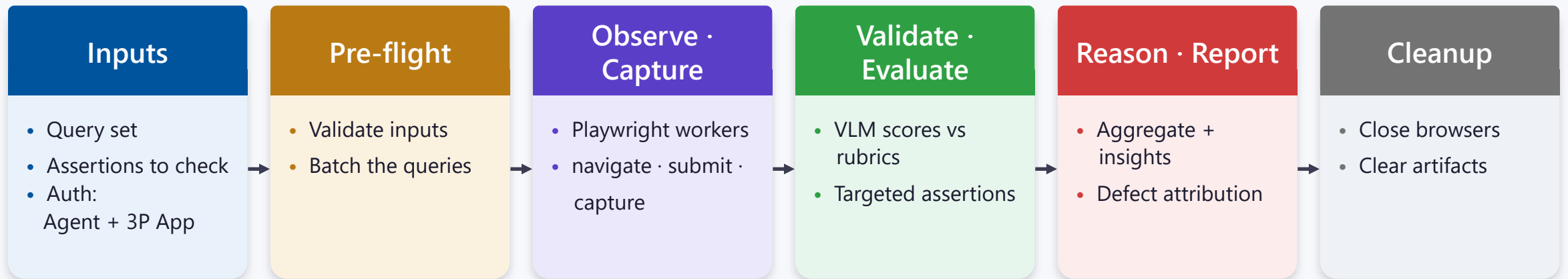


Because it models cognition, not a UI stack, the approach is **tech-stack agnostic**.

Drop it into **GitHub Copilot · Claude Code · Cursor** — it runs on that assistant's own model + **Playwright MCP**.
Your everyday coding assistant becomes a visual UX tester — zero new infrastructure.

The *workflow* at a glance

One conversational command runs the whole loop, per query, in parallel.

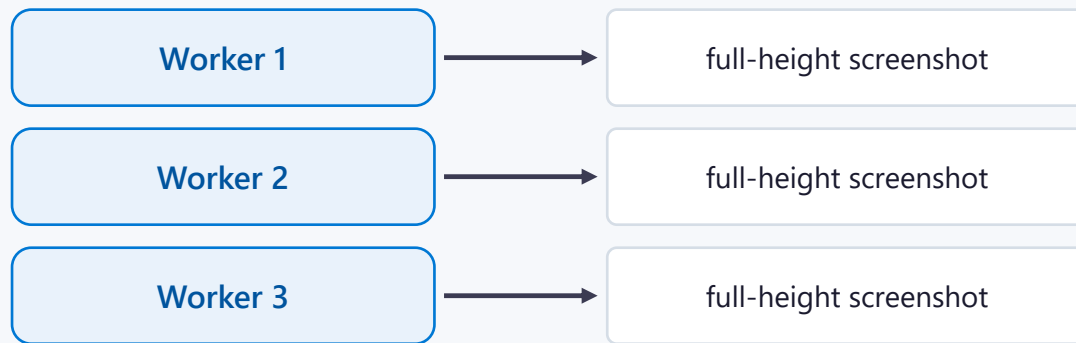


This one split gave us the biggest reliability gain

- **Deterministic helper scripts replace LLM reasoning in the capture path** — completion-polling, DOM extraction, navigation.
- **The LLM (within the coding agent) is reserved for visual judgment (& occasional course correction during navigation) only.**

Observe — capture the visual state

Parallel Playwright MCP workers



Each worker: *navigate → submit query → handle auth → capture*

Interaction, in plain language

"Click Confirm, upload the CSV,
then toggle to chart view"



Playwright MCP → ordered intents:
click · upload · toggle

No CSS selectors. No brittle scripts.

Captures both interaction triggers: **human ↔ AI chat** and **human ↔ widget** manipulation.

Validate — VLM judgment on binary criteria

Decompose rubrics into atomic yes/no checks

✓ Binary — objective

"All widget-body text uses a single font family"

"No UI element overlaps or clips another"

"Header text matches the query topic"

✗ Vague — invites sycophancy

"The typography looks good"

"The layout is clean"

"The content is relevant"

Two families of criteria — plus your own

Vision-unique

typography · alignment ·
component integrity · hierarchy

Shared (text-eval, applied visually)

structural completeness · relevance · localization

+ Pluggable (your domain)

a11y · brand · localization — drop in a judge prompt

One independent sub-agent per query → no cross-query contamination, and binary checks force objective judgment.

Validate — targeted assertions inject ground truth

Why assertions on top of rubrics?

- **Rubrics** are open-ended VLM judgment — no ground truth.
- **Assertions** encode your customers' true expectations & taste as verifiable checks — the one-way ground truth enters.

Critical

"Chart renders ≥ 3 data bars in the widget body"

Important

"Summary card shows a title and a total"

Aspirational

"Citation links are underlined in the text response"

Scope a check to a region

Full Canvas

Widget Body

Text Response

Full Canvas encompasses both.

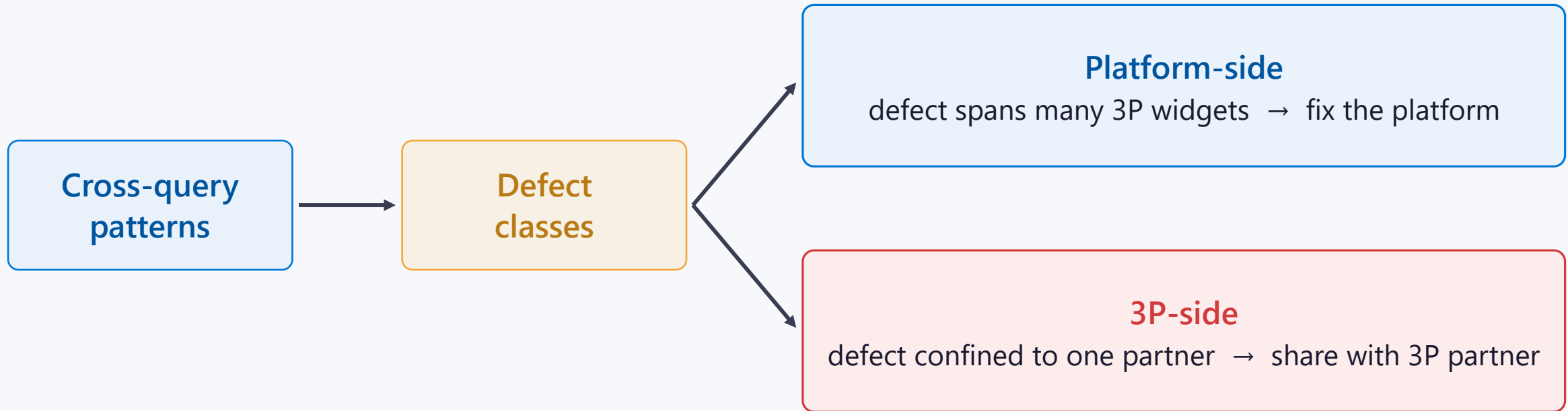
Validate — compounding multi-turn visual state



Each turn's non-deterministic output **becomes the next turn's starting state** — no ground truth, branching trajectories. We evaluate the **cumulative** canvas.

*Text benchmarks focus on the conversation; the compounding **visual** state — stacked widgets, expand/collapse, layout shifts — is essentially unaddressed.*

Reason — insights & defect attribution



Deterministic scripts aggregate the metrics

The agent writes the narrative insight — each defect gets a testable hypothesis.

Two trade-offs — and how we tame them

VLM judgment variance

The catch:

Quality scores drift slightly across runs.

Mitigation

- ✓ Binary criteria shrink open-ended judgment
- ✓ Average over multiple runs for stability

Agent path non-determinism

The catch:

The agent may navigate the platform differently each run.

Mitigation

- ✓ Deterministic helpers replace LLM navigation
- ✓ Atomic browser ops → repeatable paths

The pattern: push determinism wherever you can, and reserve the model for judgment only it can make.

In use today for MCP Apps in M365 Copilot — and beyond

Where it runs today

- **MCP App widgets in Microsoft 365 Copilot**
- **Declarative agents**
- **Federated Copilot connectors (mainline)**
- In principle: other agent surfaces

Where we're taking it next

Richer, **interaction-aware** UX evals — extending the same rigour to confirmation cards, path cards, interstitial messages, and other rich interactions. Already underway.

TAKEAWAYS

Test what the user **sees**.

1 The gap is real

- Agentic UIs compose **layout** × **content** × **client** at runtime
- Untestable by text, functional, or baseline tools.

2 Skills close it

- Encode **observe** → **validate** → **reason** as an **agent skill**
- A coding assistant becomes a baseline-free visual UX tester.

3 It generalizes

Any workflow needing tool orchestration + visual judgment + adaptive error handling — a11y, localization, consistency.

The agent builds the interface. Now something can judge it.

Thank you — questions?

That's All Folks!

For More Interesting Stuff on AI, Data Science
& the Latest Technologies...



[Tezan Sahu | LinkedIn](#)



[Low Pass Filter \(Newsletter for AI Practitioners\)](#)